

Homework 06 – Java Jukebox

Authors: Kellen, Xavier, Eric

Topics: polymorphism, exceptions, file I/O

Problem Description

Your CS 1331 TAs want to add some life to the office hours room, in the form of a Java jukebox! Unfortunately, all the TAs are depressed from the unrelenting pressure of Junior and Senior level courses. So, we have decided to delegate the task to you all! To help your TAs create this jukebox, you are to create a database that stores information about the records that can be added, played, and removed from the jukebox.

Solution Description

You will need to complete and turn in 5 classes: Record.java, JazzRecord.java, RapRecord.java, InvalidRecordException.java, and Jukebox.java.

Notes:

1. All variables should be inaccessible from other classes and require an instance to be accessed through, unless specified otherwise.
2. All methods should be accessible from everywhere and must require an instance to be accessed through, unless specified otherwise.
3. Use constructor and method chaining whenever possible! Ensure that your constructor chaining helps you reduce code reusage!
4. **Reuse code when possible.** Helper methods with appropriate visibility are optional.
5. **When throwing exceptions, a descriptive and specific message should be provided.**
6. Make sure to add Javadoc comments to your methods and classes!

Record.java

This class represents a musical record in general. Record should never be instantiated.

Variables:

- `String artist` – the name of the artist who wrote this musical piece
- `String title` – the name of the song on this record
- `String duration` – the runtime of this record
 - Represented in the format “HH:MM” (e.g., 13:31)
- `String releaseDate` – the release date of this record
 - Represented in the format “MM-YYYY” (e.g., 01-1331)
- `int timesPlayed` – the number of times this record has been played
- `char grade` – the quality grade of this record
 - Represented as 'M' for Mint, 'E' for Excellent, 'G' for Good, 'F' for Fair, and 'P' for Poor

Constructors:

- A constructor that takes in `artist`, `title`, `duration`, `releaseDate`, `timesPlayed`, and `grade`.
 - If the inputted value for `artist`, `title`, `duration`, or `releaseDate` is blank or null, throw an `IllegalArgumentException`.
 - If the inputted value for `timesPlayed` is negative, throw an `IllegalArgumentException`.
 - If the inputted value for `grade` is **NOT** 'M', 'E', 'G', 'F', or 'P', throw an `IllegalArgumentException`.
 - Leading and trailing whitespace from string inputs should be removed.

Methods:

- `toString`
 - This method should override `Object`'s `toString` method and return a `String` in the following format (single line, no curly braces, no quotation marks, no spaces preceding/following commas, no leading/trailing whitespace):

```
"{artist},{title},{duration},{releaseDate},{timesPlayed},{grade}"
```
- `equals`
 - This method should override the `equals` method from `Object`.
 - Two `Record` objects are equal if they have the same `artist`, `title`, `duration`, `releaseDate`, `timesPlayed`, and `grade`.
- Getters and setters as necessary.

JazzRecord.java

This concrete subclass of `Record` represents a jazz record.

Variables:

- `String audioEngineer` – the name of the audio engineer that mixed and mastered this jazz record

Constructors:

- A constructor that takes in `artist`, `title`, `duration`, `releaseDate`, `timesPlayed`, `grade`, and `audioEngineer`.
 - If the inputted value for `audioEngineer` is blank or null, throw an `IllegalArgumentException`.
 - Leading and trailing whitespace from string inputs should be removed.

Methods:

- `toString`
 - This method should override `Record`'s `toString` method and return a `String` in the following format (single line, no curly braces, no quotation marks, no spaces preceding/following commas, no leading/trailing whitespace):

```
"JazzRecord,{artist},{title},{duration},{releaseDate},{timesPlayed},{grade},{audioEngineer}"
```

- `equals`
 - This method should override the `equals` method from `Record`.
 - Two `JazzRecord` objects are equal if they have the same `artist`, `title`, `duration`, `releaseDate`, `timesPlayed`, `grade`, and `audioEngineer`.
- Getters and setters as necessary.

RapRecord.java

This concrete subclass of `Record` represents a rap record.

Variables:

- `String producer` – the name of the producer who arranged this musical composition

Constructors:

- A constructor that takes in `artist`, `title`, `duration`, `releaseDate`, `timesPlayed`, `grade`, and `producer`.
 - If the inputted value for `producer` is blank or null, throw an `IllegalArgumentException`.
 - Leading and trailing whitespace from string inputs should be removed.

Methods:

- `toString`
 - This method should override `Record`'s `toString` method and return a `String` in the following format (single line, no curly braces, no quotation marks, no spaces preceding/following commas, no leading/trailing whitespace):

```
"RapRecord, {artist}, {title}, {duration}, {releaseDate}, {timesPlayed}, {grade}, {producer}"
```
- `equals`
 - This method should override the `equals` method from `Record`.
 - Two `RapRecord` objects are equal if they have the same `artist`, `title`, `duration`, `releaseDate`, `timesPlayed`, `grade`, and `producer`.
- Getters and Setters as necessary.

InvalidRecordException.java

This class describes a **checked** exception that is thrown when encountering an invalid record. Note that this class should be a child of the most generic checked exception class in the `java.lang` package.

Constructors:

- A constructor that takes in a `String` representing this exception's message.
- A no-argument constructor that sets the message to "Invalid Record".

Jukebox.java

This class defines various static methods that allow a user to read and write to the jukebox. Any exceptions that you throw from within the methods of this class should be propagated. The jukebox is represented by a comma-separated value (CSV) file. We recommend that you open the CSV file in a plain text editor rather than a spreadsheet editor so you can see the delimiting commas. Please refer to the clarification thread on Ed Discussion for a sample CSV file.

Methods:

- `retrieveRecords`
 - Takes in a `String` representing the path name of the file to read from.
 - Throw a `FileNotFoundException` if the inputted path name is blank, empty, or null, or if the path name is valid but is not the path to an existing file.
 - **HINT:** Look over the File API (you might find a helpful method)!
 - If the path name represents the path to an existing file, assume the file is readable.
 - Returns an `ArrayList` of `Record` instances created from the tokens.
 - Each line of the file will contain information about a record in the jukebox.
 - File line tokens will be in the following format on each line, without curly braces:

```
{RecordType},{artist},{title},{duration},  
{releaseDate},{timesPlayed},{grade},{audioEngineer/pro  
ducer}
```
 - Iterate through the file and create a `Record` object from each token.
 - **Note:** There can be any number of lines in the file, but no lines will be empty (unless the file is empty).
 - Add each `Record` object to an `ArrayList`.
 - **HINT:** You might want to take a look at the `processInfo` method!
 - If the record type in any token is not “JazzRecord” or “RapRecord”, throw an `InvalidRecordException`. You may assume all other record details in each line are valid and well-formatted.
 - For example, an exception should be thrown given the following token:

```
"BluesRecord,Howlin' Wolf,How Many More Years,  
3:31,01-1331,35,A,cover artist"
```
- `processInfo`
 - This is a private helper method that will be used to process a line of text from the CSV into a `JazzRecord` or `RapRecord` object.
 - Takes in a `String` representing a line of info from the database to process.
 - Returns a `Record` object that is either a `JazzRecord` or `RapRecord`.
 - If the game type in the token is not “JazzRecord” or “RapRecord”, throw an `InvalidRecordException`.
 - **IMPORTANT:** Do **NOT** use a second `Scanner` object to process this line as it will break the autograder.
 - **Hint:** The `String` `split` method can be used to split a string on a specified delimiter into an array of tokens. Think about what character you can split the line apart with to separate the information.

THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.

- `stockJukebox`
 - Takes in a `String` representing the path name of the file to write to and an `ArrayList` of `Record` objects to add to the jukebox. Returns nothing.
 - If the inputted path name is blank, empty, or null, throw a `FileNotFoundException`.
 - If a file with the inputted path name does not exist, create a file with the given name.
 - You may assume the directory is writable.
 - If a file with the inputted path name exists, you may assume it is readable and does not contain empty lines (unless the file is empty).
 - Iterate through the `ArrayList` and write each record to a new line.
 - Keep the lines in the file contiguous (i.e., there should be no gaps between the lines in the file).
 - If a record has a grade of 'P', do not write it to the file (i.e., after execution of this method, the resulting file should not contain any records with a grade of 'P').
 - **Note:** If the file exists and there is already information contained in the given file, it should not be overwritten. Instead, you should add onto the end of the existing file.
 - **Hint:** How can you reuse code to read the file?
 - To implement this, you should follow a *read-modify-write* design pattern. This pattern can be used to changes the contents of a file in the following three steps:
 - Read data from a file into an intermediate data structure (e.g., an `ArrayList`).
 - Perform any necessary modifications to that data in the intermediate structure.
 - Write the data in the intermediate data structure back out to the same file.

This allows us to use familiar tools like `Scanner` and `PrintWriter` to change the contents of a file without modifying the contents of the file *directly* (which is much more difficult).
- `findRecords`
 - Takes in a `String` representing the path name of the file to read from and a `Record` object to find.
 - Throws a `FileNotFoundException` if the inputted path name is blank, empty, or null, or if the path name is valid but is not the path to an existing file.
 - If the `Record` argument is null, throw an `IllegalArgumentException`.
 - Returns an `ArrayList` of `Integer` objects.
 - Iterate through the file and find all occurrences of the specified record.
 - When you find an occurrence of the specified record, add its corresponding line number to the `ArrayList`. The first line of a file is line 0.
 - **Hint:** How can you reuse code to read the file?
 - If the inputted `Record` object is not found, throw an `InvalidRecordException`.
 - **Example:** If the specified record is found on lines 0, 3, and 9, return an `ArrayList` containing `Integers` 0, 3, and 9.

- `spinRecord`
 - Takes in a `String` representing the path name of the file to write to and a `Record` object. Returns nothing.
 - Throws a `FileNotFoundException` if the inputted path name is blank, empty, null, or if the path name is valid but is not the path to an existing file.
 - Iterate through the file and find all instances of the specified record.
 - **Hint:** How can you reuse code by calling some of the methods already written?
 - If the specified record is not found, throw an `InvalidRecordException`.
 - Spin the record by updating the state of the found instances! Updating the record consists of spinning the record and accounting for the potential damage dealt to it.
 - **Hint:** What relevant state information should be updated?
 - When spinning a record, the jukebox has a 30% chance of scratching a record. If a record is scratched, the record's grade is lowered by one level.
 - **Hint:** How could you represent the chance with a *random* value?
 - **Hint:** If the record is scratched change the grade accordingly:
 - i.e., 'M' → 'E' → 'G' → ...
 - **Note:** Each instance found matching the desired record should be spun separately, and each have their own chance to be scratched; that is to say, each record has its own grade and opportunity to be scratched. Just because one is scratched does not mean that all instances will necessarily be scratched.
 - The spun `Records` should be placed *back into the same location* that they were originally pulled from.
 - Remove all occurrences with grade of 'P' from the jukebox.
- `removeRecord`
 - Takes in a `String` representing the path name of the file to write to and a `Record` object. Returns nothing.
 - Throws a `FileNotFoundException` if the inputted path name is blank, empty, or null, or if the path name is valid but is not the path to an existing file.
 - Iterate through the file and find all instances of the specified record.
 - **Hint:** How can you reuse code by calling some of the methods already written?
 - Remove all occurrences of the specified record from the file, keeping the lines in the file contiguous (i.e., there should be no gaps between the lines in the file).
 - If the specified record is not found, throw an `InvalidRecordException`.

`Driver.java`

You will use this class to test your code.

Methods:

- `main`
 - Create two `JazzRecord` and two `RapRecord` objects.
 - Write the objects into a file called "TestRecords.csv" using `stockJukebox`.
 - **Note:** It is recommended to open the CSV file using a text editor rather a spreadsheet editor when viewing or editing the file.
 - Create another `JazzRecord` object and add it to "TestRecords.csv".
 - Read the CSV file into an `ArrayList` using `retrieveRecords`. Print each object to a new line.
 - Call `playRecord` on a record in the CSV file to update each occurrence from the file.

**THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.**

- Call `removeRecord` on a record currently in the CSV file to remove each occurrence from the file.
- **Note:** This is to help you get started with testing your code, and it is not comprehensive. It is *strongly* recommended that you create more test cases.

Checkstyle

You must run Checkstyle on your submission (to learn more about Checkstyle, check out [cs1331-style-guide.pdf](#) under the Checkstyle Resources module on Canvas). **The Checkstyle cap for this assignment is 35 points.** This means there is a maximum point deduction of 35. If you don't have Checkstyle yet, download it from Canvas → Modules → Checkstyle Resources → `checkstyle-8.28.jar`. Place it in the same folder as the files you want to run Checkstyle on. Run Checkstyle on your code like so:

```
$ java -jar checkstyle-8.28.jar YourFileName.java
Starting audit...
Audit done.
```

The message above means there were no Checkstyle errors. If you had any errors, they would show up above this message, and the number at the end would be the number of points we would take off (limited by the Checkstyle cap). In future assignments we will be increasing this cap, so get into the habit of fixing these style errors early!

Additionally, you must Javadoc your code.

Run the following to only check your Javadocs:

```
$ java -jar checkstyle-8.28.jar -j yourFileName.java
```

Run the following to check both Javadocs and Checkstyle:

```
$ java -jar checkstyle-8.28.jar -a yourFileName.java
```

For additional help with Checkstyle see the [CS 1331 Style Guide](#).

Turn-In Procedure

Submission

To submit, upload the files listed below to the corresponding assignment on Gradescope:

- `Record.java`
- `JazzRecord.java`
- `RapRecord.java`
- `InvalidRecordException.java`
- `Jukebox.java`

Make sure you see the message stating the assignment was submitted successfully. From this point, Gradescope will run a basic autograder on your submission as discussed in the next section. **Any autograder tests are provided as a courtesy to help “sanity check” your work and you may not see all the test cases used to grade your work.** You are responsible for thoroughly testing your submission on your own to ensure you have fulfilled the requirements of this assignment. If you have questions about the requirements given, reach out to a TA or Professor via the class forum for clarification.

**THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.**

You can submit as many times as you want before the deadline, so feel free to resubmit as you make substantial progress on the assignment. We will only grade your latest submission. **Be sure to submit every file each time you resubmit.**

Gradescope Autograder

If an autograder is enabled for this assignment, you may be able to see the results of a few basic test cases on your code. Typically, tests will correspond to a rubric item, and the score returned represents the performance of your code on those rubric items only. If you fail a test, you can look at the output to determine what went wrong and resubmit once you have fixed the issue. **We reserve the right to hide any or all test cases, so you should make sure to test your code thoroughly against the assignment's requirements.**

The Gradescope tests serve two main purposes:

- Prevent upload mistakes (e.g., non-compiling code)
- Provide basic formatting and usage validation

In other words, the test cases on Gradescope are by no means comprehensive. Be sure to thoroughly test your code by considering edge cases and writing your own test files. You also should avoid using Gradescope to compile, run, or Checkstyle your code; you can do that locally on your machine.

Other portions of your assignment can also be graded by a TA once the submission deadline has passed, so the output on Gradescope may not necessarily reflect your grade for the assignment.

Burden of Testing

You are responsible for thoroughly testing your submission against the written requirements to ensure you have fulfilled the requirements of this assignment.

Be **very careful** to note the way in which text output is formatted and spelled. Minor discrepancies could result in failed autograder cases.

If you have questions about the requirements given, reach out to a TA or Professor via the class forum for clarification.

Allowed Imports

- `java.io.File`
- `java.io.FileNotFoundException`
- `java.io.PrintWriter`
- `java.util.ArrayList`
- `java.util.Scanner`

Feature Restrictions

There are a few features and methods in Java that overly simplify the concepts we are trying to teach or break our autograder. For that reason, do not use any of the following in your final submission:

- `var` (the reserved keyword)
- `System.exit`
- `System.arraycopy`

Collaboration

Only discussion of the assignment at a conceptual high level is allowed. You can discuss course concepts and assignments broadly; that is, at a conceptual level to increase your understanding. If you find yourself dropping to a level where specific Java code is being discussed, that is going too far. Those discussions should be reserved for the instructor and TAs. To be clear, you should never exchange code related to an assignment with anyone other than the instructor and TAs.

The only code you may share are test cases written in a Driver class. If you choose to share your Driver class, they should be posted to the assignment discussion thread on the course discussion forum. We encourage you to write test cases and share them with your classmates, but we will not verify their correctness (i.e., use them at your own risk).

Important Notes (Don't Skip)

- Non-compiling files will receive a 0 for all associated rubric items.
- Do not submit `.class` files.
- Test your code in addition to the basic checks on Gradescope.
- Submit every file each time you resubmit.
- Read the "Allowed Imports" and "Restricted Features" to avoid losing points.
- **Check on Ed Discussion for a note containing all official clarifications and sample outputs.**

It is expected that everyone will follow the Student-Faculty Expectations document and the Student Code of Conduct. The professor expects a **positive, respectful, and engaged academic environment** inside the classroom, outside the classroom, in all electronic communications, on all file submissions, and on any document submitted throughout the duration of the course. **No inappropriate language is to be used**, and any assignment deemed by the professor to contain inappropriate offensive language or threats will get a zero. You are to use professionalism in your work. Violations of this conduct policy will be turned over to the Office of Student Integrity for misconduct.